

FLEXIBLE ARCHITECTURE

유연한 설계의 기술: 다형성 (Polymorphism)

"같은 메시지, 다른 동작"

Chapter 7. Polymorphism

1. 다형성(Polymorphism)이란?

"하나의 이름(Interface)으로 여러 가지 형태의 동작을 수행하는 능력"



Poly (많은) + Morph (형태)

겉모습은 하나인데,
속은 천차만별이다.

- ✓ 같은 메서드 호출
- ✓ 다른 동작
- ✓ 다른 결과!

2. 핵심 메커니즘: 오버라이딩 (Overriding)

"부모가 물려준 기능을 내 방식으로 재정의(덮어쓰기)합니다."

✂ 일반 캐릭터 (Parent)

```
def attack():  
    print("주먹 치기")
```

➡ 상속

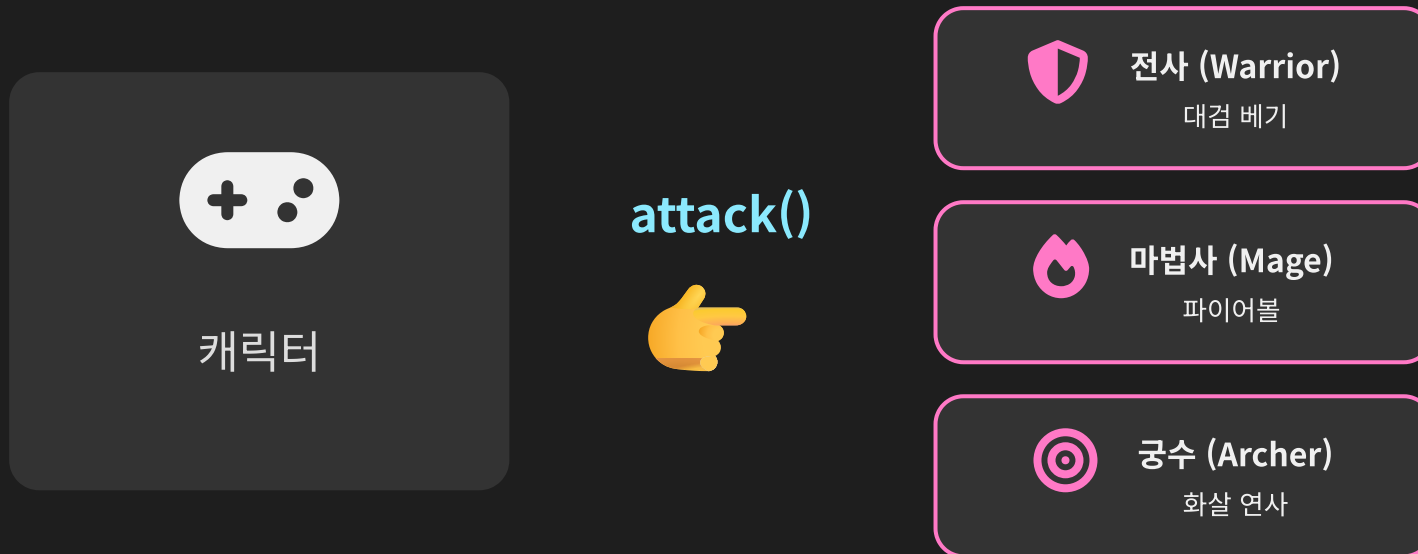
🔥 마법사 (Child)

```
def attack():  
    print("파이어볼 발사!")
```

"이름은 같지만 기능은 업그레이드!"

3. 비유: 캐릭터의 공격

"캐릭터의 **공격**만 호출하면 됩니다.
캐릭터가 무엇이든 상관없습니다."



4. 절차지향 vs 객체지향 (코드의 변화)

"if-else 지옥"에서 탈출하여 다형성의 천국으로!"

😓 절차지향 (Bad)

새 직업이 나오면 코드를 또 고쳐야 함

```
def attack(character):  
    if character.type == "Warrior":  
        print("대검 베기!")  
    elif character.type == "Mage":  
        print("파이어볼!")  
    elif character.type == "Archer":  
        print("화살 쏘!")  
    # 직업 추가될 때마다 if문 계속 추가...
```

😎 객체지향 (Good)

코드를 전혀 수정할 필요 없음!

```
def attack(character):  
    # 다형성: 캐릭터가 누구든 알아서 공격  
    character.attack()  
  
# 새로운 직업(Ninja)이 나와도?  
# 이 함수는 건드릴 필요 없음.  
# Ninja 클래스만 잘 만들어서 넣으면 됨.
```

5. 다형성의 진짜 효과: 확장성

"기존 코드를 건드리지 않고(Closed), 새로운 기능을 추가(Open)할 수 있습니다."

(OCP: 개방-폐쇄 원칙)



Plug & Play

USB 포트처럼 규격(메서드)만
맞으면
무엇이든 꽂아서 바로 쓸 수
있습니다.



유연한 유지보수

기능을 추가할 때 메인 로직을
수정할 필요가 없으므로 버그가
줄어듭니다.

4-2. 파이썬의 다형성: 오리 타이핑 (Duck Typing)

"파이썬은 상속 없이도 다형성이 가능합니다."

duck_typing.py

```
# 상속(Inheritance)을 받지 않았음!
class Dog:
    def speak(self): print("멍멍!")

class Cat:
    def speak(self): print("야옹~")

class Robot:
    def speak(self): print("삐리리~")

# 객체들이 서로 달라도 'speak'만 할 줄 알면 OK
animals = [Dog(), Cat(), Robot()]

for a in animals:
    a.speak()
```



Java와 다른 점

- **Java:** **Animal**을 상속받아야만 다형성 가능 (엄격함)
- **Python:** 상속 관계가 없어도 **메서드 이름**만 같으면 실행됨 (유연함)



"Duck Typing"

"오리처럼 걷고 꺽꺽거리면,
그것은 오리다."

요약: 유연한 설계를 위하여

- 🎯 오버라이딩: 부모의 기능을 내 입맛대로 바꾼다.
- 😬 다형성: 하나의 명령으로 다양한 객체를 조종한다.
- 🔧 효과: **if-else**를 제거하고, **확장하기 쉬운** 코드를 만든다.